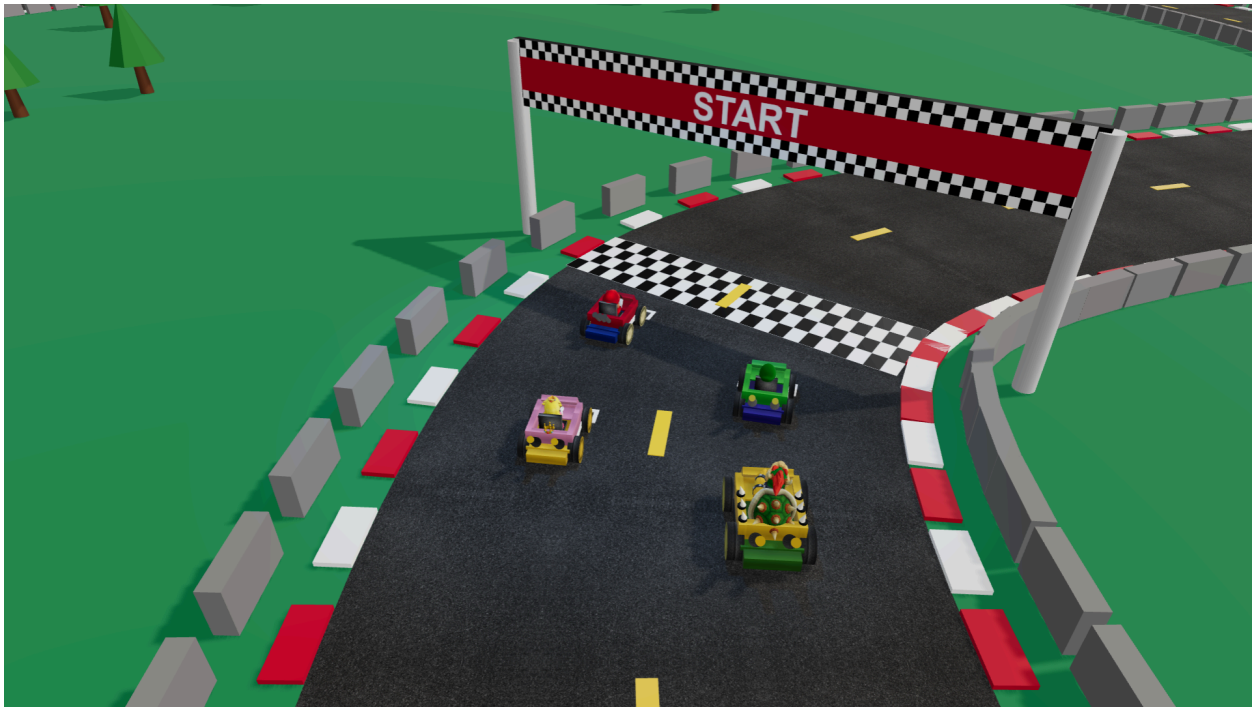
Relatório da disciplina de Computação Gráfica - Mario Kart 3D (Entrega 4)

O atual relatório apresenta o que foi alterado da Entrega 3 para a Entrega 4. Nesta versão, o projeto ganhou texturas em karts, personagens, pista e obstáculos, com as coordenadas UV de cada superfície tratadas à mão quando a geometria não era pronta. As câmeras e as três fontes de luz da entrega anterior foram mantidas, e agora é possível ver como a luz age sobre as superfícies texturizadas.



[Figura 1: vista de largada (tecla 1) com as texturas aplicadas, mostrando o asfalto, a lataria dos karts, os pneus e os cones perto do grid.]

1. A cena 3D

A cena continua montada em "src/main.js" e reúne a pista ("Track"), os pilotos ("Racer"), os obstáculos e o "CameraManager" e o "LightController" da entrega anterior. A estrutura do loop de animação não mudou. O que foi alterado é o material de quase tudo que aparece na tela:

- Foi criada a função "loadTexture" ("src/utils/loadTexture.js"), que carrega uma imagem de "public/textures/" com o "TextureLoader" e já marca a textura como sRGB. Ela é utilizada pelos karts e pelo asfalto. Os obstáculos têm um carregador próprio, "loadObstacleTexture", que guarda cada SVG num "Map" para que as 3 pilhas de pneus, por exemplo, compartilhem a mesma textura em vez de carregar o arquivo de novo.
- Os karts ("Kart.js") deixaram de ser apenas uma cor sólida, e agora a lataria, banco e pneus usam imagens. O logo do capô é desenhado num canvas e convertido em textura ("kartEmblems.js").

- Os personagens em ".gltf" passaram por um reparo de texturas no "Character.js", pois alguns modelos do Sketchfab vieram sem parte dos mapas.
- A pista ganhou o asfalto fotográfico, bem como uma faixa de largada quadriculada e um letreiro do pórtico. Estes dois últimos foram desenhados em canvas.
- O grid passou a ter quatro pilotos (Mario, Luigi, Peach e Bowser) e existem três pistas, trocadas com a tecla **Enter**. Cada pista tem o seu filtro de cor sobre a mesma foto de asfalto, o que está explicado na seção 4.
- A paisagem ("src/scenery/") foi adicionada com peças de primitivas. Ela usa materiais com cor, "flatShading" e, na cidade, material emissivo. Não usa imagens, então não conta como textura nesta entrega (ver seção 6).

2. Aplicação de múltiplas texturas

O projeto usa mais de três texturas diferentes, em objetos de tipos diferentes:

Textura	Origem	Onde é aplicada
"TexturaLatariaCarroSuave.jpg"	imagem	carroceria, capô, flancos e chassi dos karts
"BancoCarro.webp"	imagem	assento e encosto
"PneuLateral.jpg" e "RodaCarro.jpg"	imagem	banda de rodagem e tampas das rodas
"texturaAsfalto.avif"	imagem	asfalto das três pistas
"cone-hazard.svg"	SVG	base e corpo dos cones
"barrel-wood.svg"	SVG	barris
"tire-tread.svg"	SVG	pilhas de pneus
"oil-slick.svg"	SVG	manchas de óleo
logo do capô (letra, Bowser ou coroa)	canvas 2D	capô de cada kart
"?" da caixa de item	canvas 2D	caixas de item
xadrez e letreiro "START"	canvas 2D	faixa de largada e pórtico
"_diffuse.png", "_normal.png"	arquivos dos modelos	Mario, Luigi, Peach e Bowser

As imagens dos karts e do asfalto foram tiradas da internet. Os quatro SVGs dos obstáculos foram feitos pelo grupo, pensando no estilo de desenho animado do jogo: cone laranja com faixa clara, barril vermelho de aduelas, pneu de borracha escura com losangos e óleo roxo/azulado com reflexo irisado.

Sobre a coerência com a proposta visual, a lataria é uma cópia clareada da imagem original (misturada 50% com branco). A textura é multiplicada pela cor do material, então a mesma imagem serve para o

kart vermelho do Mario, o verde do Luigi, o rosa da Peach e o amarelo do Bowser, e o desenho metálico não pesa mais que a cor do personagem.

Nos personagens, "Character._patchMissingMaps()" percorre os materiais do modelo e, para os que vieram sem "map", procura na pasta de texturas os arquivos "<nome do material>_diffuse.png", depois "_normal.png" e por último "_specularGlossiness.png". Cada arquivo é carregado uma vez só (há um cache por URL) e, se não existir, o material segue como estava.



[Figura 2: os quatro karts com seus pilotos no grid, mostrando a mesma lataria nas quatro cores.]

3. Mapeamento de texturas e coordenadas UV

O Three.js já traz as coordenadas UV nas geometrias prontas ("BoxGeometry", "CylinderGeometry", "LatheGeometry", "TorusGeometry", "CircleGeometry"), sempre normalizadas em [0, 1]. Cada vértice recebe um par (u, v) que diz qual ponto da imagem cai nele, e o fragment shader lê a cor com "texture2D()". Neste projeto as UVs aparecem de três formas.

UV pronta da geometria.

- Barril: a "LatheGeometry" gira o perfil em volta do eixo Y; u percorre a volta e v sobe da base ao topo. O SVG tem as aduelas na vertical e duas faixas escuras perto de v = 0,22 e v = 0,78, que coincidem com a altura dos aros cinza do modelo (0,24 e 0,81 da altura de 1,05).
- Pneu do kart: a "CylinderGeometry" tem 3 grupos de faces (lateral, tampa de cima e tampa de baixo), então o mesmo mesh recebe um array de materiais, a banda de rodagem na lateral e a roda nas tampas.
- Cone: o corpo é um tronco de cone e a imagem cobre a volta inteira.
- Pilha de pneus: na "TorusGeometry", u corre ao longo do anel e v ao redor do tubo.

- Mancha de óleo e logos: a "CircleGeometry" mapeia o círculo inscrito no quadrado da imagem, então o desenho circular do SVG e do canvas encaixa sem sobra.
- Caixa de item e letreiro: cada face da "BoxGeometry" recebe a imagem inteira. No letreiro, a ordem das faces (+x, -x, +y, -y, +z, -z) foi usada para colocar a textura só nas duas faces grandes e deixar as outras pretas.

UV calculada à mão (asfalto). A malha da pista é montada vértice a vértice em "Track._buildAsphalt()", então as UVs também são calculadas lá. Para cada ponto da curva há dois vértices, esquerdo e direito, com:

- u = 0 na esquerda e u = "trackWidth / 4" na direita (3 cópias da foto na pista de 12 unidades, 3,5 na de 14);
- v = "u_curva · comprimento / 4", que conta quantas cópias da foto já couberam ao longo da pista.

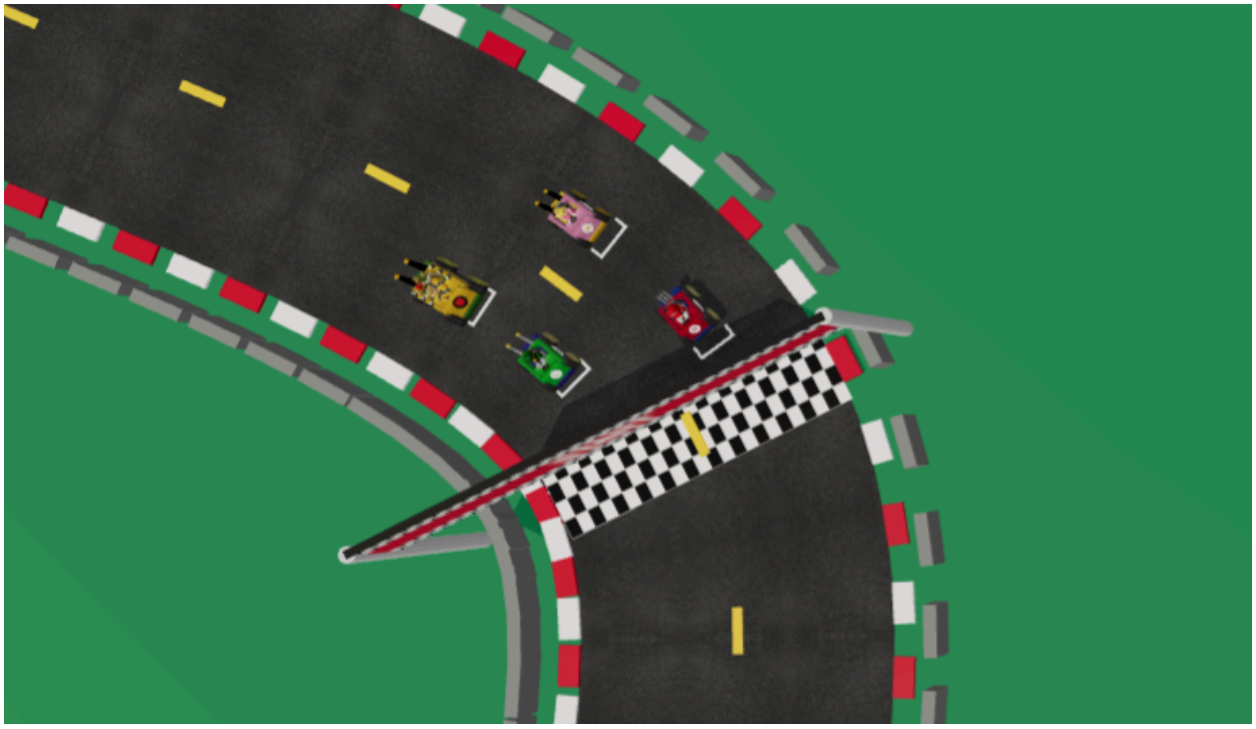
Com o tamanho da cópia fixo em 4 unidades ("ASPHALT_TILE"), o grão do asfalto tem o mesmo tamanho nas três pistas, mesmo com larguras e comprimentos diferentes. Valores acima de 1 fazem a textura repetir, e usamos "MirroredRepeatWrapping", que espelha cada cópia e faz as bordas se encontrarem.

UV ajustada com "repeat" e "offset".

- Banda de rodagem: na imagem o desenho está em pé, mas na lateral do cilindro u dá a volta no pneu. A textura é girada 90° ("rotation = π/2") e repetida 3 vezes ("repeat.set(1, 3)").
- Roda de lado: "RodaCarro.jpg" (600 x 600 px) tem três desenhos. As tampas usam só o quadrado da roda, que começa em x = 181 px, termina em y = 428 px e tem 393 px de lado. Isso vira "repeat = 393/600" (cerca de 0,655) e "offset = (181/600, 1 - 428/600)", com o y invertido pois o offset é medido de baixo para cima.
- Luigi: o material do olho direito não tinha PNG. Como o lado direito é o espelho do esquerdo, a textura do par foi clonada com "repeat.x = -1" e "offset.x = 1", o que faz u virar 1 - u ("Character._mirrorMaterialMap()").



[Figura 3: rodas dos karts de perto, com a banda de rodagem na lateral e o desenho da roda recortado na tampa.]



[Figura 4: o asfalto visto de cima (tecla 2) e de perto na largada, mostrando que não aparecem emendas entre as cópias.]

Para não deixar as texturas distorcidas, foram feitos estes ajustes:

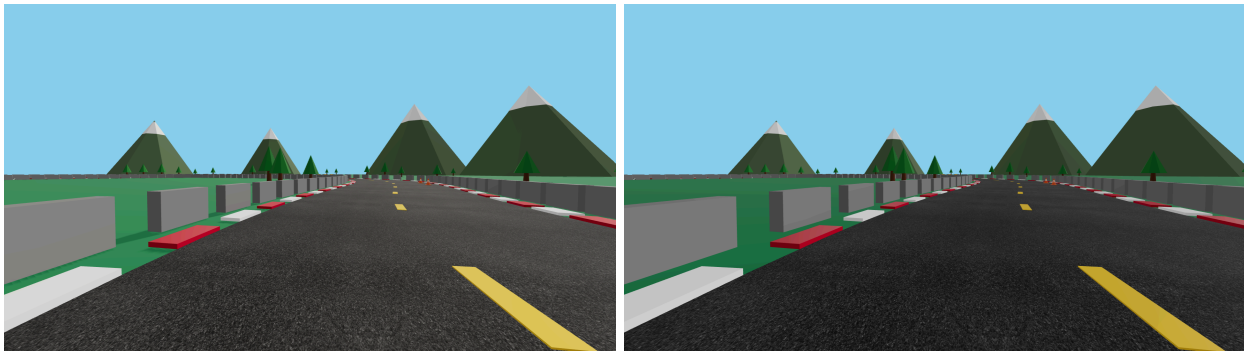
- nos modelos em ".gltf", "flipY = false", que é a convenção do formato (o padrão do "TextureLoader" inverte a imagem);
- "RepeatWrapping" nas texturas dos personagens, porque algumas meshes do Luigi (o branco do olho direito) têm UV maior que 1 e, no modo "clamp", só pegavam a borda transparente e sumiam;
- meshes com escala negativa (determinante da matriz menor que 0) têm a ordem dos vértices invertida em "_fixMirroredMeshes()", para os triângulos não ficarem virados para dentro e a textura aparecer do lado certo.

4. Integração com a iluminação

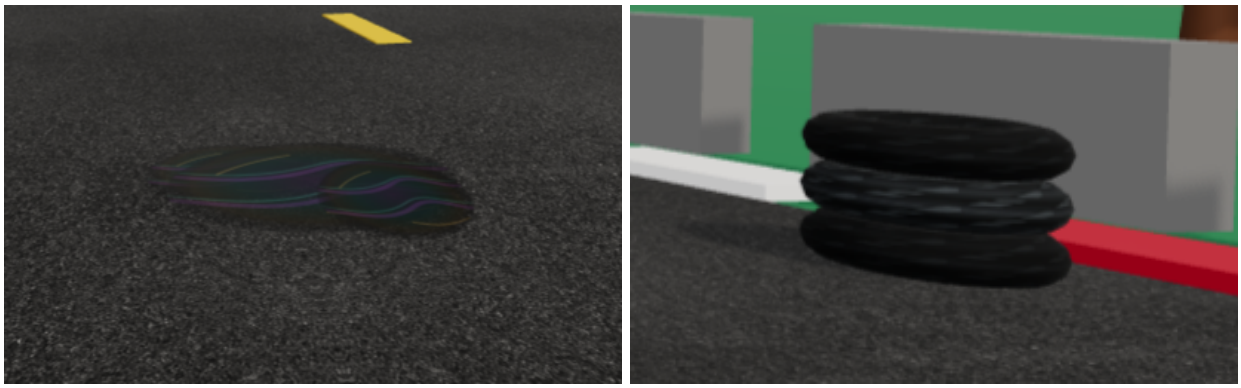
- As três fontes da entrega anterior foram mantidas sem mudança de intensidade: "ambient" (0,3), "sun" (1,2, com sombras) e "trackSpot" (azul, 2,0, sobre a largada). Todos os materiais com textura são "MeshStandardMaterial", então a cor final de cada pixel é a cor da textura vezes a cor do material, iluminada pelas três luzes e pelo mapa de ambiente (cuja intensidade está em 0,3 no "main.js"). Foi isso que deu para observar:
- **Asfalto:** a cor do material funciona como filtro sobre a foto (branco no Circuito Tradicional, bege no Deserto, azul no Neon). Com "roughness 0,8", o brilho do sol é fraco e espalhado, e o holofote azul tinge o asfalto em volta da largada.

- **Lataria:** com "roughness 0,4", o capô e os flancos ganham um brilho do sol, e a textura aparece por baixo dele. Nas laterais, a face virada para o sol fica mais clara e a oposta cai para a luz ambiente.
- **Pneus e rodas:** a banda é quase fosca ("roughness 0,9") e o aro tem "roughness 0,6" e cor do preset. Os desenhos continuam visíveis na parte sombreada porque a ambiente impede o preto total.
- **Mancha de óleo:** "roughness 0,12" e "metalness 0,4". É o material mais brilhante da cena, e o reflexo do ambiente e do sol mexe quando a câmera se move. Como é semitransparente e fica rente ao chão, usa "depthWrite = false" e "polygonOffset", para não piscar sobre o asfalto.
- **Sombras:** os obstáculos e os karts projetam sombra sobre o asfalto texturizado. Os cones e barris com "castShadow" deixam sombras nítidas sobre a foto.
- **Cidade à noite:** os prédios e postes usam cor emissiva, então brilham sem depender de luz, enquanto o asfalto azulado e os karts reagem ao holofote.

Para separar o efeito de cada fonte, as teclas **L** (ambiente), **K** (sol) e **0** (holofote) ligam e desligam as luzes. Só com a ambiente as texturas aparecem chapadas, sem volume. Só com o sol surgem os brilhos e as sombras. Só com o holofote só o círculo azul da largada é iluminado. Com as três desligadas a cena ainda se enxerga por causa do mapa de ambiente, como já descrito na entrega anterior.



[Figura 5: a mesma cena com todas as luzes ligadas e com só a luz ambiente (tecla K e 0 desligadas).]



[Figura 6: óleo e pneus de perto, com o brilho do sol e a sombra projetada sobre o asfalto.]

5. Principais dificuldades

- O asfalto começou com a grade das emendas visível, porque a foto repetia sem espelhar. Resolvemos com "MirroredRepeatWrapping". Depois ele virava um borrão visto de longe e de lado, e a filtragem anisotrópica ("anisotropy = 8") resolveu.
- A textura original da lataria era forte demais e brigava com as cores dos karts. Criamos uma cópia clareada ("TexturaLatariaCarroSuave.jpg"), o que deixou a cor do personagem mais importante que o desenho metálico. Como cada face de caixa recebe a imagem inteira, faces de tamanhos diferentes esticam a textura de formas diferentes, e a versão mais suave esconde isso.
- A imagem da roda tem três desenhos e, sem o recorte, a tampa do cilindro mostrava os três espremidos. Foi preciso medir o quadrado da roda em pixels e converter para "repeat" e "offset", lembrando que o offset em y vai de baixo para cima.
- Os modelos do Sketchfab vieram com materiais sem mapa (o Luigi) ou com UV fora de [0, 1]. Por isso o reparo é feito por código, em vez de editar os modelos.
- Na pilha de pneus, o quadrado do SVG é esticado no anel: o contorno do anel (raio 0,36) é cerca de 2,6 vezes maior que o do tubo (raio 0,14), então os losangos ficam alongados ao longo do pneu. Como o padrão é diagonal e o objeto é pequeno, o efeito é discreto, mas existe.
- Óleo e asfalto brigavam na mesma altura (z-fighting). Isso foi resolvido com "polygonOffset" no material do óleo e uma pequena diferença de altura entre os discos.

6. O que ainda falta implementar

- Texturizar as peças da paisagem. Árvores, montanhas, cactos, pedras, platôs e prédios usam só cor e "flatShading". Os platôs e os prédios seriam os primeiros candidatos.
- Registrar a origem das imagens dos karts e do asfalto em "public/CREDITS.md", que hoje lista só os modelos 3D.
- Cobrir o circuito inteiro com as sombras do sol. A câmera de sombra da luz direcional continua com o tamanho padrão do Three.js, então as sombras só aparecem perto da largada.
- Faróis nos karts, com o "update(kartPosition)" do "LightController" já preparado.
- Movimento de verdade. O modo demo só leva os pilotos por uma curva fixa; falta controle por teclado, aceleração, colisão e a lógica de corrida.
- Itens funcionais, som e adversários com comportamento próprio.

Créditos

Os modelos 3D de Mario, Luigi, Peach e Bowser continuam creditados em "public/CREDITS.md" (licença CC-BY 4.0). Os karts, a pista, os obstáculos e os SVGs de textura foram feitos pelo grupo.